# LIBRARY MANAGEMENT SYSTEM

**A PROJECT REPORT**

*Submitted by*

**THIRUKUMARAN T**
(2116240701568)

*in partial fulfilment of the End Semester Examination for the course*

**CB23F35 –**
**LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR**

*in*

**[BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY]**
**[DEPARTMENT / BRANCH NAME]**




**RAJALAKSHMI ENGINEERING COLLEGE**
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

**NOVEMBER 2026**

**RAJALAKSHMI ENGINEERING COLLEGE**

(An Autonomous Institution, Affiliated to Anna University, Chennai)

# BONAFIDE CERTIFICATE

Certified that this project report titled "**LIBRARY MANAGEMENT SYSTEM**" is the bonafide work of **THIRUKUMARAN T** (2116240701568), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

**SIGNATURE**

Dr. J. Manoranjini
**HEAD OF THE DEPARTMENT**
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

**SIGNATURE**

Dr. J. Jinu Sophia
**Assistant Professor**
COURSE FACULTY
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held on

_____.

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

# DECLARATION

I, THIRUKUMARAN T (2116240701568), hereby declare that the project report titled "**LIBRARY MANAGEMENT SYSTEM**", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, dashboard, Deluge scripts and workflows — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. The AI-assisted application builder (Zia) used to generate the starting application, and every other template, code snippet or external resource used in this work, has been duly acknowledged in the report.

**SIGNATURE OF THE STUDENT**
THIRUKUMARAN T
240701568

Place: Chennai

Date:_____

# ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan **and** Vice-Chairman Mr. M. Abhay Shankar for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S.N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinu Sophia, Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

**THIRUKUMARAN T**

# ABSTRACT

Libraries in educational institutions must keep track of books, student members, issues, returns and due dates. When these records are maintained in registers or separate spreadsheets, available stock is not always up to date, overdue books are noticed late, and the librarian cannot obtain totals or reports quickly. This leads to duplicate entries, lost time and disputes over fines.

This project presents a Library Management System developed on the Zoho Creator low-code platform. The application was started with Zia AI app creation and then refined through Creator forms, reports and Deluge workflows. It contains four forms — Book Details, Student Details, Book Issue and Book Return — connected through lookup fields and Auto Number record IDs. Five table reports (Book Details, Student Details, Book Issue, Book Return and Overdue Books) provide search and filtering. A deliberately plain Admin Dashboard shows Total Books, Total Students, Books Issued and Available Books, with a Recent Transactions table. Deluge scripts check stock before every issue, reduce and restore Available Quantity, block duplicate returns, validate quantities and dates, and calculate overdue fines once the administrator has configured a per-day fine rate.

The application was verified with a ten-point test checklist covering stock reduction, zero-stock blocking, duplicate-return blocking, overdue filtering and dashboard totals. Because every figure is read from the database, the librarian gets accurate stock, issue status and overdue information without manual calculation, and the cloud-hosted application can be reached from any browser.



**Keywords:** *Low-Code Development; Zoho Creator; Zia; Deluge Scripting; Workflow Automation; Library Management*

# PROJECT AT A GLANCE

| Attribute | Details |
|---|---|
| Project Title | Library Management System |
| Application Domain | Education – Library Management |
| Student Name & Register No. | THIRUKUMARAN T – 240701568 |
| Zoho Creator Edition | Education / Free / Standard / Professional |
| Application Name (Link Name) | Library_Management_System |
| Published / Live URL | https://creatorapp.zoho.in/thirukumaran.t.2024.cse_r396/library-management-system |
| Demo Credentials (per role) | Administrator:application owner login (Zoho account thirukumaran2529); credentials are provided at the time of the demonstration |
| Number of Forms / Subforms | 6 forms / 0 subforms |
| Number of Reports (by type) | List: 5 (Book Details, Student Details, Book Issue, Book Return, Overdue Books) |
| Number of Pages / Dashboards | 1 (Admin Dashboard) |
| Number of Workflows | Form: 5 (Book Details, Book Issue and Book Return events); Schedule / Approval / Blueprint / Batch: 0 |
| Deluge Functions / Custom Actions | 5 form-event scripts (DS-01 to DS-05) |
| Integrations | Zoho Mail (e-mail delivery Deluge sendmail); Zoho Zia prediction (AI fields) |
| Roles & Profiles Defined | Administrator (CEO); permission profiles: Admin, Manager and User. |
| Environments Used | Development workspace of Zoho Creater, published at the live URL above |
| Date of Production Deployment | Published and accessible through the live URL |

# TABLE OF CONTENTS


**1 INTRODUCTION**

1.1 Background ..... 1
1.2 Problem Statement ..... 1
1.3 Objectives ..... 1
1.4 Scope of the Project ..... 2
1.5 Justification for a Low-Code Approach ..... 2
1.6 Organisation of the Report ..... 2

**2 LITERATURE SURVEY AND SYSTEM STUDY ..... 3**

2.1 Study of the Existing System ..... 3
2.2 Limitations of the Existing System ..... 3
2.3 Low-Code versus Traditional Development ..... 3
2.4 Literature Survey ..... 4
2.5 Emerging Trends in Low-Code Development ..... 5
2.6 Proposed System ..... 5

**3 REQUIREMENT ANALYSIS ..... 6**

3.1 Stakeholders and User Roles ..... 6
3.2 Functional Requirements ..... 6
3.3 Non-Functional Requirements ..... 6
3.4 Hardware and Software Requirements ..... 6
3.5 Use Case Model ..... 6
3.6 Business Rules, Assumptions and Constraints ..... 7

**4 SYSTEM DESIGN ..... 9**

4.1 System Architecture ..... 9
4.2 Data Model ..... 9
4.3 Form Design ..... 10
4.4 Process and Workflow Design ..... 11
4.5 User Interface and Navigation Design ..... 11
4.6 Access Control Design ..... 11
4.7 Design Considerations ..... 12

**5 IMPLEMENTATION ..... 13**

5.1 Application Setup ..... 13
5.2 Forms and Data Capture ..... 13
5.3 Reports ..... 14
5.4 Pages and Dashboards ..... 15
5.5 Deluge Scripting ..... 15
5.6 Workflows and Automation ..... 17
5.7 Integrations ..... 18
5.8 User Management and Role-Based Access ..... 18
5.9 AI and Analytics Features ..... 18
5.10 Pro-Code Extension ..... 19
5.11 Mobile and Portal Access ..... 19

**6 TESTING AND DEBUGGING ..... 20**
6.1 Testing Strategy ..... 20
6.2 Test Cases ..... 20
6.3 Debugging Deluge Scripts ..... 21
6.4 Defect Log ..... 21
6.5 User Acceptance Testing ..... 21
6.6 Test Data and Boundary Tests ..... 21
**7 DEPLOYMENT, SECURITY AND GOVERNANCE ..... 23**
7.1 Environments and Deployment Strategy ..... 23
7.2 Performance Optimisation ..... 23
7.3 Security Features ..... 23
7.4 Governance and Administration ..... 24
**8 RESULTS AND DISCUSSION ..... 25**
8.1 Application Output ..... 25
8.2 Achievement of Objectives ..... 25
8.3 Comparison with the Existing System ..... 26
8.4 Limitations of the Low-Code Approach ..... 26
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum ..... 26
**9 CONCLUSION AND FUTURE ENHANCEMENTS ..... 27**
9.1 Conclusion ..... 27
9.2 Future Enhancements ..... 27
9.3 Learning Outcomes and Career Relevance ..... 27
**REFERENCES ..... 28**
**APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST . 29**
**APPENDIX B – COURSE OUTCOME MAPPING ..... 30**
**APPENDIX C – COMPLETE DELUGE SOURCE CODE ..... 31**
**APPENDIX D – USER MANUAL ..... 33**
**APPENDIX E – EVALUATION RUBRIC ..... 35**
**APPENDIX F – VIVA-VOCE RECORD SHEET ..... 36**

## LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application ..... 4
Table 2.2: Summary of Literature Survey ..... 4
Table 2.3: Limitations Addressed by the Proposed System ..... 5
Table 2.4: Modules of the Proposed System ..... 5
Table 3.1: Stakeholders and User Roles ..... 6
Table 3.2: Functional Requirements ..... 6
Table 3.3: Non-Functional Requirements ..... 6
Table 3.4: Hardware and Software ..... 6
Requirements Table 3.5: Use Case ..... 7
Description – Issue a Book Table 3.6: ..... 7
Use Case Description – Return a Book
Table 3.7: Business Rules ..... 7
Table 4.1: Entities and Relationships ..... 9
Table 4.2: Field Specification – Book Details Form ..... 10
Table 4.3: Field Specification – Student Details Form ..... 10
Table 4.4: Field Specification – Book Issue Form ..... 10
Table 4.5: Field Specification – Book Return Form ..... 10
Table 4.6: Validation Rules ..... 10
Table 4.7: Application Navigation Menu ..... 11
Table 4.8: Role–Permission Matrix ..... 11
Table 4.9: Fine Calculation Design ..... 12
Table 4.10: Design Decisions and Rationale ..... 12
Table 5.1: Reports Implemented ..... 14
Table 5.2: Deluge Scripts Implemented ..... 16
Table 5.3: Workflows Implemented ..... 17
Table 5.4: Integrations Implemented ..... 18
Table 6.1: Test Cases ..... 20
Table 6.2: Defect Log ..... 21
Table 6.3: User Acceptance Feedback ..... 21
Table 6.4: Sample Book Records for Testing ..... 21
Table 6.5: Sample Student Records for Testing ..... 22
Table 6.6: Boundary and Negative Test Conditions ..... 22
Table 7.1: Deployment Checklist ..... 23
Table 7.2: Performance Optimisation Techniques ..... 23
Table 7.3: Security Controls Implemented ..... 23
Table 8.1: Scenario with Sample Data ..... 25
Table 8.2: Mapping of Objectives to Outcomes ..... 25
Table 8.3: Existing System versus Proposed Application ..... 26
Table A.1: Feature Coverage Checklist ..... 29
Table B.1: Mapping of Course Outcomes to Report Chapters ..... 30
Table D.1: Messages Shown by the Application ..... 33
Table E.1: Evaluation Rubric for the Individual Project ..... 35
Table F.1: Viva-Voce Questions and Responses ..... 36
Table F.2: Marks Summary ..... 36

## LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing System ..... 3
Figure 3.1: Use Case Diagram of the Application ..... 7
Figure 4.1: System Architecture of the Application ..... 9
Figure 4.2: Entity–Relationship Diagram ..... 9
Figure 4.3: Workflow for Issuing and Returning a Book ..... 11
Figure 4.4: Navigation Structure of the Application ..... 11
Figure 4.5: Wireframe of the Book Issue Form ..... 12
Figure 5.1: Application Home and Navigation Sections ..... 13
Figure 5.2: Book Details Form in the Form Builder ..... 13
Figure 5.3: Book Details Form in Live Mode ..... 14
Figure 5.4: Student Details Form in Live Mode ..... 14
Figure 5.5: Book Issue Form in Live Mode ..... 14
Figure 5.6: Book Return Form in Live Mode ..... 14
Figure 5.7: Book Details Report ..... 14
Figure 5.8: Overdue Books Report ..... 15
Figure 5.8a: Student Details Report ..... 15
Figure 5.8b: Students by Status (Kanban View) ..... 15
Figure 5.8c: Books by Category (Kanban View) ..... 15
Figure 5.9: Admin Dashboard Layout with Sample Data (plain, no charts) ..... 15
Figure 5.10: Admin Dashboard Page ..... 15
Figure 5.11: Workflow Configuration ..... 18
Figure 5.12: Roles and Permission Profile Configuration ..... 18
Figure 5.13: Zia AI Application Creation in Use ..... 19
Figure 5.14: Application on Mobile ..... 19
Figure 6.1: Deluge Debug Output ..... 21
Figure 7.1: Publish to Production ..... 23
Figure 7.2: Audit Trail and Administration Settings ..... 24
Figure 8.1: Scenario Step 1 – Issue a Book ..... 25
Figure 8.2: Scenario Step 2 – Return the Book ..... 25
Figure 8.3: Scenario Step 3 – Dashboard Totals ..... 25

## LIST OF ABBREVIATIONS

| | |
|---|---|
| **API** | Application Programming Interface |
| **CO** | Course Outcome |
| **CRUD** | Create, Read, Update, Delete |
| **DS** | Deluge Script |
| **ER** | Entity–Relationship |
| **FR / NFR** | Functional / Non-Functional Requirement |
| **ID** | Identifier |
| **ISBN** | International Standard Book Number |
| **LCAP** | Low-Code Application Platform |
| **LMS** | Library Management System |
| **RBAC** | Role-Based Access Control |
| **REST** | Representational State Transfer |
| **TC** | Test Case |
| **UAT** | User Acceptance Testing |
| **UC** | Use Case |
| **UI** | User Interface |
| **Zia** | Zoho's AI assistant |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background

A library is the academic backbone of every engineering college. The library considered in this project lends books in four categories — Programming, Database, Networking and General — to students of the CSE, IT, ECE and EEE departments across the first to the fourth year of study. Each title may be held in several copies, so the librarian must know at all times how many copies exist and how many are currently on the shelf.

In day-to-day operation the librarian records every new book, registers every student member, notes each issue with its due date and enters each return, charging a fine when a book comes back late. These records are the only source of truth for stock, so their accuracy directly decides whether a student can borrow the book that is needed.

This project digitises the work as a Library Management System on Zoho Creator [2], a low-code platform in which data forms, reports and automation are configured visually and extended with Deluge scripts [3].

## 1.2 Problem Statement

Handling books, students, issues and returns in registers or in separate spreadsheets is slow and error-prone. Because nothing links a student to a book automatically, stock figures drift away from reality and the librarian has no quick view of what is overdue. The specific pain points are listed below.

- Available copies are not updated reliably after each issue and return.
- Overdue books and fines are found only by manually checking dates.
- Student, book and transaction records are scattered, with no instant totals or searchable reports.
- Nothing stops a book being issued with zero copies, or the same return being entered twice.

## 1.3 Objectives

- To design forms and lookup relationships for books, students, issues and returns (Chapters 4 and 5).
- To automate stock updates, validation and fine calculation using Deluge workflows (Sections 5.5 and 5.6).
- To provide table reports with search and filters, and a plain admin dashboard (Sections 5.3 and 5.4).
- To build the application quickly using Zia AI app creation in Zoho Creator (Section 5.9).
- To verify the application using a structured test checklist and deploy it with basic governance controls (Chapters 6 and 7).

### 1.4 Scope of the Project

**In scope:** four forms (Book Details, Student Details, Book Issue, Book Return), five table reports, one Admin Dashboard page and the issue and return workflows. The application is used by the library administrator on the web or the Zoho Creator mobile app, in the default Zoho Creator appearance.

**Out of scope:** a student login portal, online fine payment, barcode scanning, SMS or email reminders, and integration with other Zoho or external services. These are listed as future enhancements in Chapter 9.

### 1.5 Justification for a Low-Code Approach

A library application is records-driven: it needs forms, relationships, validations and reports rather than complex algorithms. Low-code platforms suit this kind of problem because forms, lookups and reports are configured visually, hosting, backup and security are built in, and Deluge is needed only for the stock and fine logic [1], [2]. Zia can also generate a working starting application from a text prompt, which shortens development time and lets a non-specialist maintain the result.

### 1.6 Organisation of the Report

Chapter 2 reviews the existing system and related literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements. Appendices A to F give the feature checklist, course outcome mapping, complete Deluge source code, user manual, evaluation rubric and viva record.

# CHAPTER 2
# LITERATURE SURVEY AND SYSTEM STUDY

## 2.1 Study of the Existing System

In the existing process the librarian keeps a register or a spreadsheet. When a student asks for a book, the librarian checks the shelf, writes the student and book details by hand, notes a due date on a slip and hands over the book. On return the entry is located again, the return date is written and, if the book is late, the fine is worked out manually. Stock is corrected only when someone remembers to update the count.

The study walked through the borrowing cycle and noted each point at which a record is written, read or corrected. Three observations stand out. First, the same student and book names are written repeatedly, so spellings differ and a student's history cannot be traced reliably. Second, the only link between a loan and the stock figure is the librarian's memory. Third, the due date sits on a slip separate from the record, so overdue books are discovered only when someone happens to look.




**Figure 2.1:** As-Is Process Flow of the Existing System

## 2.2 Limitations of the Existing System

- Stock counts depend on manual updates, so mistakes go unnoticed until a student finds the shelf empty.
- Overdue books and fines require manual comparison of dates and are often missed.
- There is no instant search, no totals and no ready-made reports for the administrator.
- Records of one student or book are hard to trace because entries are free text.
- A book can be issued with no copies left, or the same return recorded twice.

## 2.3 Low-Code versus Traditional Development

Table 2.1 compares the two approaches for this particular application.

**Table 2.1:** Comparison of Low-Code and Traditional Development for the Proposed Application

| Criterion | Traditional (Pro-Code) | Low-Code on Zoho Creator |
|---|---|---|
| Development time | Separate UI, database schema and server code must be written and integrated | Forms and reports built visually; Zia generates a starting application |
| Skills required | Full-stack programming and database administration | Form design and basic Deluge scripting |
| Hosting and infrastructure | Own or rented server, deployment and backups | Built-in Zoho cloud hosting and backups |
| Mobile application | Separate mobile app must be developed | Zoho Creator mobile app available by default |
| Security and access control | Login, roles and sessions coded by the developer | Built-in login, roles and permissions |
| Maintenance and change | Code changes, testing and redeployment | Edit a form or script and publish |
| Customisation limits | Unlimited | Bound by platform features; pro-code extension possible |
| Cost | Developer time and server cost | Platform subscription or education access |

For this application the largest saving comes from not having to build the data layer, login, hosting and mobile access, which together dominate the effort of a traditional implementation. The cost of the low-code route is that the interface follows the platform's default look and complex logic must fit Deluge. Both are acceptable here, because the project itself asks for the default Zoho Creator appearance and the logic is limited to stock and fine rules.

## 2.4 Literature Survey

Table 2.2 summarises the sources reviewed for this project; full details are in the References.

**Table 2.2:** Summary of Literature Survey

| S.No. | Author(s) / Source and Year | Title / Product | Key Findings | Relevance to this Project |
|---|---|---|---|---|
| 1 | W. Rosam and O. Kalnichevski [1] | Low-Code Development: A Practical Guide | Explains low-code concepts and the move from citizen to professional developer | Conceptual basis for choosing low-code |
| 2 | Zoho Corporation [2] | Zoho Creator Help Documentation | Describes forms, reports, pages, workflows and deployment | Primary platform reference |
| 3 | Zoho Corporation [3] | Deluge Script Reference Guide | Defines the Deluge syntax for fetch, update and validation | Used to write DS-01 to DS-05 |
| 4 | R. Waszkowski, 2019 [4] | Low-code platform for automating business processes | Uses a low-code platform to automate business processes | Shows suitability for process automation |
| 5 | A. Sahay et al., 2020 [5] | Comparison of low-code development platforms | Compares features of low-code platforms | Supports the choice of Zoho Creator |
| 6 | Koha Community [6] | Koha Library Software | Open-source integrated library system with circulation and catalogue | Reference for library features |

Taken together, the sources show that low-code platforms are mature enough for process-oriented, data-centric applications [4], [5]; that the platform documentation [2], [3] is sufficient for one developer to build and script such an application; and that established library systems [6] share the core entities of catalogue, member, loan and return that this project models. The survey therefore supports four linked forms and

automation of only the rules that protect stock integrity.

### 2.5 Emerging Trends in Low-Code Development

Three trends influenced the design. **AI-assisted app building**: Zia generated the starting forms and reports from a prompt, and the effort moved from building to verifying. **Citizen development**: a librarian-level user can maintain the forms and reports without programming. **Hyper-automation**: routine rules such as stock reduction and duplicate-return blocking are automated so that the administrator only selects records.

### 2.6 Proposed System

The proposed Zoho Creator application replaces manual registers with four linked forms. Table 2.3 shows how it overcomes each limitation in Section 2.2.

**Table 2.3:** Limitations Addressed by the Proposed System

| Limitation | How the Proposed System Overcomes It |
|---|---|
| Stale stock counts | Available Quantity is changed automatically by DS-02 and DS-04 |
| Manual overdue checking | Overdue Books report filters active issues past the due date |
| No search or totals | Search, filters and a dashboard of database totals |
| Free-text records | Lookup fields connect students, books and issues |
| Invalid or duplicate transactions | Validation scripts DS-01, DS-03 and DS-05 block them |

The proposed system is organised into the modules summarised in Table 2.4. Each module maps to a menu section, so the administrator finds every task in one place.

**Table 2.4:** Modules of the Proposed System

| Module | Components | Benefit to the Administrator |
|---|---|---|
| Book management | Book Details form and report | Accurate title, category and copy counts |
| Student management | Student Details form and report | Single record per student with department and year |
| Circulation | Book Issue and Book Return forms and reports | Issue and return with automatic stock control and fines |
| Monitoring | Overdue Books report and Admin Dashboard | Immediate view of overdue items and library totals |

# CHAPTER 3
# REQUIREMENT ANALYSIS

## 3.1 Stakeholders and User Roles

**Table 3.1:** Stakeholders and User Roles

| Role | Description | Key Responsibilities in the Application |
|---|---|---|
| Administrator | Librarian who manages the library records | Maintain books and students, issue and return books, view reports and dashboard, and set the fine rate |
| Student | Library member (record only, no login) | Details are held in Student Details and linked to issues; receives books and returns them |

Only the Administrator logs in to the application. Students appear as data records so that every issue can be traced to a person.

## 3.2 Functional Requirements

**Table 3.2:** Functional Requirements

| FR ID | Requirement | Role | Priority |
|---|---|---|---|
| FR-01 | Add, view, edit, delete and search book records with total and available quantity | Admin | High |
| FR-02 | Maintain student records (name, department, year, email, phone) | Admin | High |
| FR-03 | Issue a book through student and book lookups only if Available Quantity > 0, and reduce it by one | Admin | High |
| FR-04 | Reject a Due Date earlier than the Issue Date, and negative or excess quantities | Admin | High |
| FR-05 | Return a book, set status Returned, increase Available Quantity by one and block duplicate returns | Admin | High |
| FR-06 | Calculate the overdue fine per day once the administrator sets the fine rate | Admin | Medium |
| FR-07 | Provide five table reports with search and filters, including Overdue Books | Admin | High |
| FR-08 | Show Total Books, Total Students, Books Issued, Available Books and recent transactions on a plain dashboard | Admin | Medium |
| FR-09 | Keep all forms and reports reachable from the navigation menu | Admin | Medium |

## 3.3 Non-Functional Requirements

**Table 3.3:** Non-Functional Requirements

| NFR ID | Category | Requirement |
|---|---|---|
| NFR-01 | Usability | Default Zoho Creator appearance, plain white background, simple navigation |
| NFR-02 | Data integrity | Lookups instead of typed names; all totals and reports read from the database |
| NFR-03 | Security | Only the logged-in administrator can access the application |
| NFR-04 | Availability | Cloud-hosted and reachable from any modern browser or the mobile app |
| NFR-05 | Maintainability | Forms, reports and scripts can be edited and republished by a low-code developer |
| NFR-06 | Data safety | Existing records and working forms are never deleted or overwritten by changes |

## 3.4 Hardware and Software Requirements

**Table 3.4:** Hardware and Software Requirements

| Item | Specification |
|---|---|
| Client device | Any PC, laptop, tablet or smartphone with internet access |
| Browser | Current Chrome, Edge, Firefox or Safari |
| Platform | Zoho Creator with Zia AI app creation |
| Scripting | Deluge (form events and workflows) |
| Hosting | Zoho cloud (no local server required) |

## 3.5 Use Case Model

Figure 3.1 shows the use cases of the single actor. Return a Book extends to Calculate Overdue Fine only when the return is late and a rate is set.

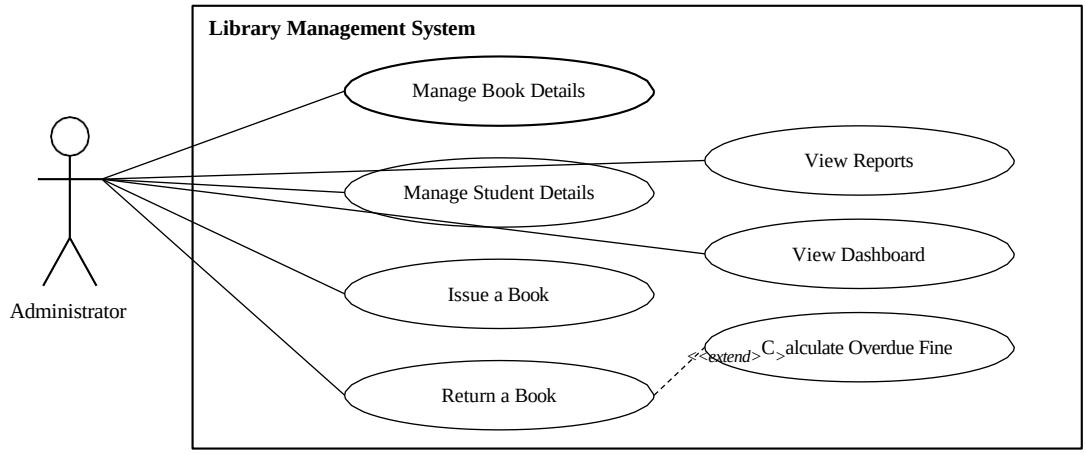



**Figure 3.1:** Use Case Diagram of the Application

**Table 3.5:** Use Case Description – Issue a Book

| | |
|---|---|
| Use Case ID | UC-01 |
| Use Case Name | Issue a Book |
| Actor(s) | Administrator |
| Pre-condition | Student and book records exist; administrator is logged in |
| Main Flow | 1. Open Book Issue form. 2. Select Student and Book by lookup. 3. Enter Issue Date and Due Date. 4. Submit. 5. System saves the issue, sets status Issued and reduces Available Quantity by one. |
| Alternate Flow | If Available Quantity is zero or Due Date is before Issue Date, the system shows a message and cancels the submission. |
| Post-condition | Issue record stored; stock reduced by one |
| Related FR | FR-03, FR-04 |

**Table 3.6:** Use Case Description – Return a Book

| | |
|---|---|
| Use Case ID | UC-02 |
| Use Case Name | Return a Book |
| Actor(s) | Administrator |
| Pre-condition | An active issue record exists |
| Main Flow | 1. Open Book Return form. 2. Select the active Issue Record. 3. Enter Return Date. 4. Submit. 5. System restores stock, marks the issue Returned and applies any fine. |
| Alternate Flow | If the issue is already returned, the system blocks the submission. |
| Post-condition | Return stored; Available Quantity increased by one |
| Related FR | FR-05, FR-06 |

## 3.6 Business Rules, Assumptions and Constraints

The business rules in Table 3.7 come from the project requirements and are enforced by the validation scripts of Chapter 5.

**Table 3.7:** Business Rules

| Rule ID | Business Rule | Enforced By |
|---|---|---|
| BR-01 | Book quantities must never be negative | DS-05 |
| BR-02 | Available Quantity must not exceed Total Quantity | DS-05 |
| BR-03 | A book cannot be issued when Available Quantity is zero | DS-01 |
| BR-04 | Due Date must not be earlier than Issue Date | DS-01 |
| BR-05 | Issuing a book reduces Available Quantity by one | DS-02 |
| BR-06 | A return must refer to a valid issue that is not already returned | DS-03 |
| BR-07 | Returning a book increases Available Quantity by one | DS-04 |
| BR-08 | A fine is charged only after a per-day rate has been configured | DS-04 |

**Assumptions.**

- One copy is issued per transaction and each issue is returned as a whole.
- The library administrator enters all records; students do not use the system.
- Dates are entered in the application's default date format.

**Constraints.**

- The interface must keep the default Zoho Creator appearance and a plain dashboard.
- The per-day fine rate is not specified and must be approved by the project owner.
- Existing records and working forms must not be deleted or overwritten.

# CHAPTER 4
# SYSTEM DESIGN

## 4.1 System Architecture

The application follows Zoho Creator's layered model (Figure 4.1). The administrator reaches the application through a web browser or the mobile app. The application layer holds the forms, reports, dashboard page and Deluge workflows, and every record is stored in Creator's cloud database. Zia is used only at build time to generate the starting application.

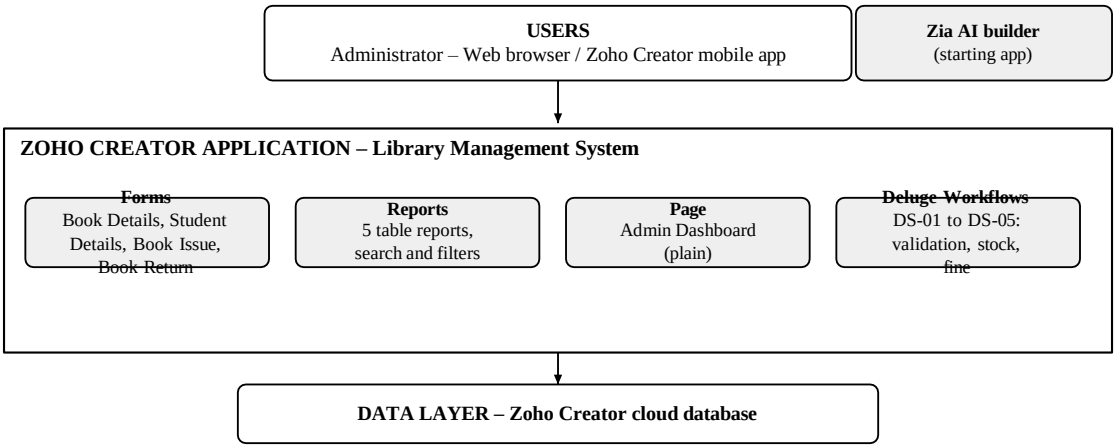



**Figure 4.1:** System Architecture of the Application

## 4.2 Data Model

Student_Details and Book_Details are master forms. Book_Issue links one student to one book through two lookups, and Book_Return links to exactly one issue record (Figure 4.2, Table 4.1).

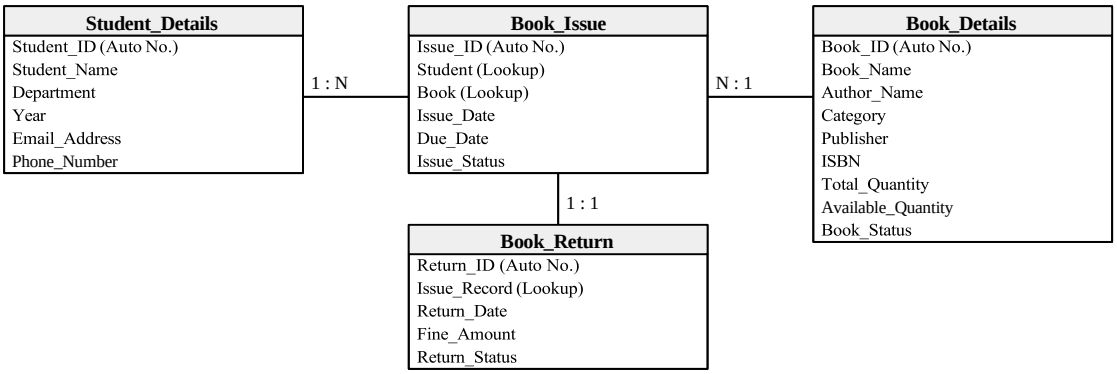



**Figure 4.2:** Entity–Relationship Diagram

**Table 4.1:** Entities and Relationships

| Entity (Form) | Key Fields | Related To | Relationship | Implemented Using |
|---|---|---|---|---|
| Student_Details | Student_ID, Student_Name | Book_Issue | One-to-Many | Lookup in Book_Issue |
| Book_Details | Book_ID, ISBN | Book_Issue | One-to-Many | Lookup in Book_Issue |
| Book_Issue | Issue_ID | Book_Return | One-to-One | Lookup in Book_Return |

### 4.3 Form Design

Tables 4.2 to 4.5 give the field specification of each form using Zoho Creator field type names. Auto Number fields give each record a unique ID, and important fields are mandatory.

**Table 4.2:** Field Specification – Book Details Form

| Field Label | Link Name | Field Type | Mand. | Validation / Remarks |
|---|---|---|---|---|
| Book ID | Book_ID | Auto Number | — | Unique record identifier |
| Book Name | Book_Name | Single Line | Yes | |
| Author Name | Author_Name | Single Line | No | |
| Category | Category | Drop Down | No | Programming, Database, Networking, General |
| Publisher | Publisher | Single Line | No | |
| ISBN | ISBN | Single Line | No | Optional unique-value check |
| Total Quantity | Total_Quantity | Number | Yes | Not negative |
| Available Quantity | Available_Quantity | Number | Yes | Not negative; not above Total Quantity |
| Book Status | Book_Status | Drop Down | No | Available / Out of Stock |

**Table 4.3:** Field Specification – Student Details Form

| Field Label | Link Name | Field Type | Mand. | Validation / Remarks |
|---|---|---|---|---|
| Student ID | Student_ID | Auto Number | — | Unique record identifier |
| Student Name | Student_Name | Single Line | Yes | |
| Department | Department | Drop Down | No | CSE, IT, ECE, EEE |
| Year | Year | Drop Down | No | 1, 2, 3, 4 |
| Email Address | Email_Address | Email | No | Email format check |
| Phone Number | Phone_Number | Phone | No | |

**Table 4.4:** Field Specification – Book Issue Form

| Field Label | Link Name | Field Type | Mand. | Validation / Remarks |
|---|---|---|---|---|
| Issue ID | Issue_ID | Auto Number | — | System generated |
| Student | Student | Lookup | Yes | Student Details |
| Book | Book | Lookup | Yes | Book Details |
| Issue Date | Issue_Date | Date | Yes | Default: today |
| Due Date | Due_Date | Date | Yes | Not earlier than Issue Date |
| Issue Status | Issue_Status | Drop Down | Yes | Issued / Returned / Overdue |

**Table 4.5:** Field Specification – Book Return Form

| Field Label | Link Name | Field Type | Mand. | Validation / Remarks |
|---|---|---|---|---|
| Return ID | Return_ID | Auto Number | — | System generated |
| Issue Record | Issue_Record | Lookup | Yes | Active Book Issue records only |
| Return Date | Return_Date | Date | Yes | |
| Fine Amount | Fine_Amount | Currency / Decimal | No | Overdue days × per-day rate |
| Return Status | Return_Status | Drop Down | Yes | Returned / Completed |

**Table 4.6:** Validation Rules

| Rule | Form | Enforced By |
|---|---|---|
| Quantity must not be negative | Book Details | DS-05 |
| Available Quantity must not exceed Total Quantity | Book Details | DS-05 |
| Due Date must not be earlier than Issue Date | Book Issue | DS-01 |
| A book cannot be issued when Available Quantity is zero | Book Issue | DS-01 |
| A return must refer to an issue that is not yet returned | Book Return | DS-03 |
| Student and book chosen by lookup, never typed | Issue / Return | Lookup fields |

## 4.4 Process and Workflow Design

Figure 4.3 shows the two automated processes. Validation runs on submit and can cancel the record; stock update runs after the record is saved.

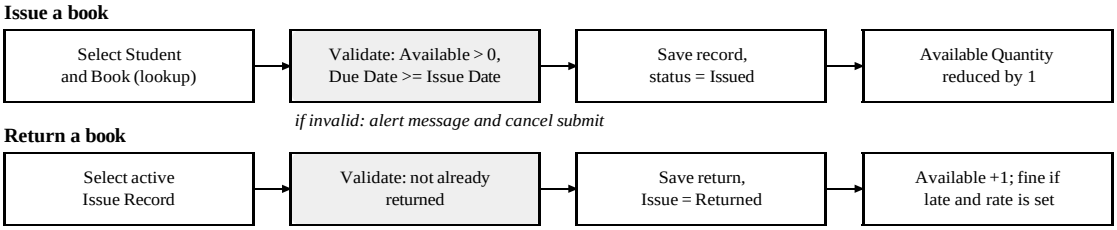



**Figure 4.3:** Workflow for Issuing and Returning a Book

## 4.5 User Interface and Navigation Design

The default Zoho Creator navigation menu is kept. Figure 4.4 shows the menu structure and Table 4.7 the purpose of each section. No custom colours, charts or tiles are used.

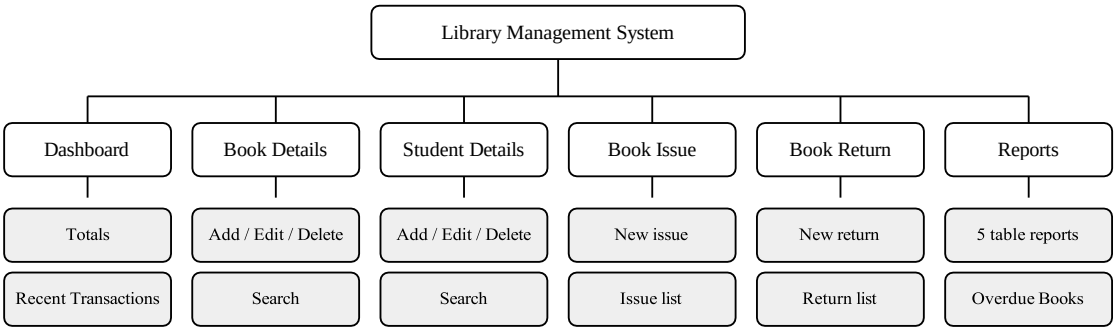



**Figure 4.4:** Navigation Structure of the Application

**Table 4.7:** Application Navigation Menu

| Menu | Purpose |
|---|---|
| Dashboard | Basic library totals and recent transactions |
| Book Details | Add, view, edit, delete and search book records |
| Student Details | Maintain student records |
| Book Issue | Create and track issue transactions |
| Book Return | Record returned books and applicable fines |
| Reports | Open the required table-based reports |

## 4.6 Access Control Design

A = Add, E = Edit, V = View, D = Delete, — = no access. Students are data records and have no login.

**Table 4.8:** Role–Permission Matrix

| Component | Administrator | Student (no login) |
|---|---|---|
| Form: Book Details | A E V D | — |
| Form: Student Details | A E V D | — |
| Form: Book Issue | A E V D | — |
| Form: Book Return | A E V D | — |
| Reports (all five) | V | — |
| Page: Admin Dashboard | V | — |

## 4.7 Design Considerations

Three design decisions shaped the data model and workflows. Quantities are stored on the book record, so every report and the dashboard read one value instead of recounting loans. The issue status is stored on the issue record, so the duplicate-return check needs a single comparison. The fine is calculated at return time, from a rate the administrator controls, so rates can change without editing scripts. Figure 4.5 shows the wireframe of the Book Issue form and Table 4.9 the fine rules.

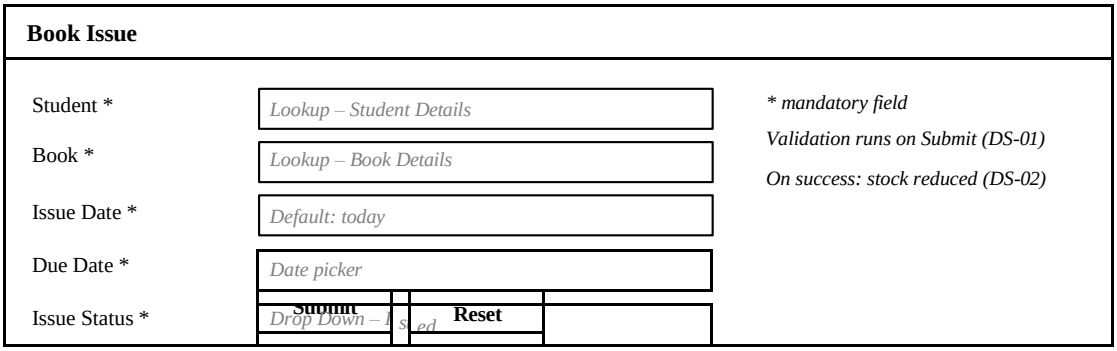



**Figure 4.5:** Wireframe of the Book Issue Form

**Table 4.9:** Fine Calculation Design

| Condition at Return | Fine Amount |
|---|---|
| Returned on or before the Due Date | 0 |
| Late, but no per-day rate configured | 0 (no fine until the rate is approved) |
| Late, and rate configured | Late days × per-day rate (for example, 5 days late × rate) |

**Table 4.10:** Design Decisions and Rationale

| Decision | Rationale |
|---|---|
| Store Available Quantity on the book record | Reports and dashboard read one value; no recounting |
| Store Issue Status on the issue record | Duplicate-return check is one comparison |
| Use lookups for student, book and issue | Prevents mistyped names and mismatched records |
| Plain default appearance | Matches the project requirement; easy to maintain |

# CHAPTER 5
# IMPLEMENTATION

## 5.1 Application Setup

The application *Library Management System* was created in Zoho Creator with the Zia AI app builder and then checked and corrected manually. The menu holds Dashboard, Book Details, Student Details, Book Issue, Book Return and Reports. The default Zoho Creator theme is used, with no custom colours, in line with the plain design requirement.

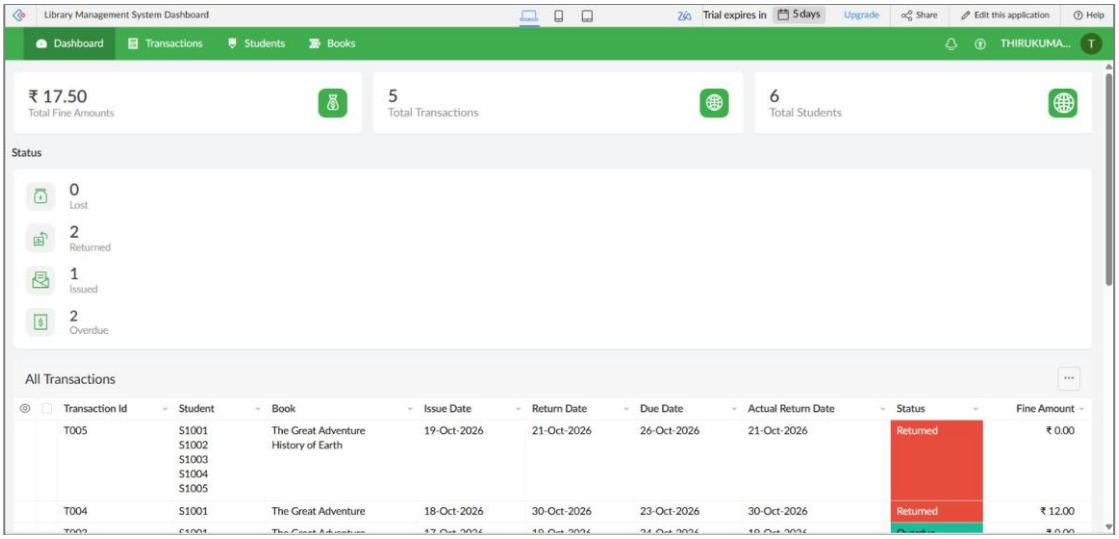



**Figure 5.1:** Application Home and Navigation Sections

## 5.2 Forms and Data Capture

Nine field types are used: Auto Number, Single Line, Drop Down, Number, Email, Phone, Date, Currency/Decimal and Lookup. **Book Details** stores each title with total and available copies. **Student Details** stores department, year and contact details. **Book Issue** uses Student and Book lookups, with Issue Date defaulting to today and a mandatory Due Date. **Book Return** uses an Issue Record lookup limited to active issues. Names are never typed, so records cannot be mismatched.

Form properties were set for a clean data-entry experience:

- A success message confirms each saved record, and the Submit and Reset buttons use their default labels.
- Issue Date defaults to today; Issue Status defaults to Issued; Book Status is set by the stock scripts.
- Lookup fields display the student name and book name, so the administrator never sees internal IDs.

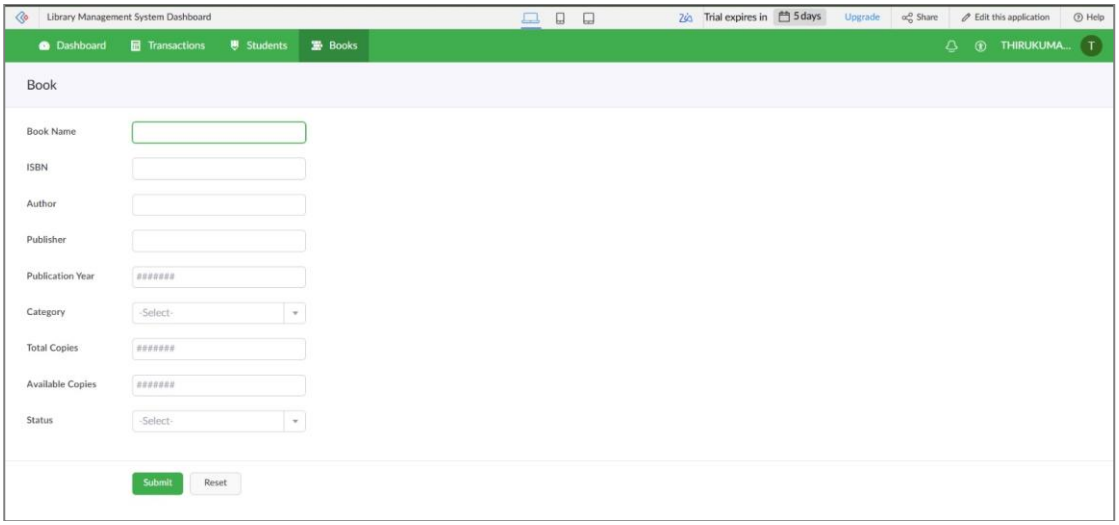



**Figure 5.3:** Book Details Form in Live Mode

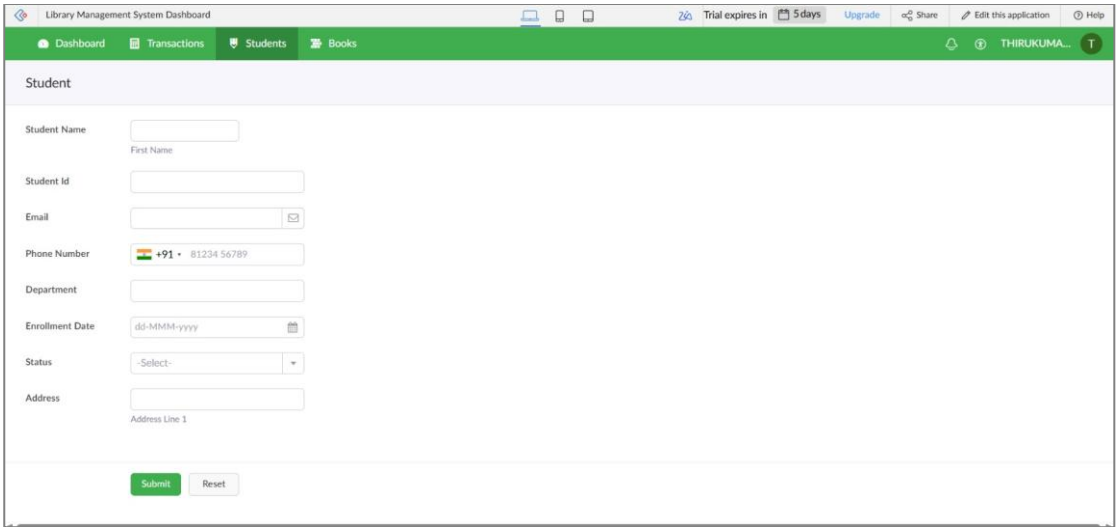



**Figure 5.4:** Student Details Form in Live Mode

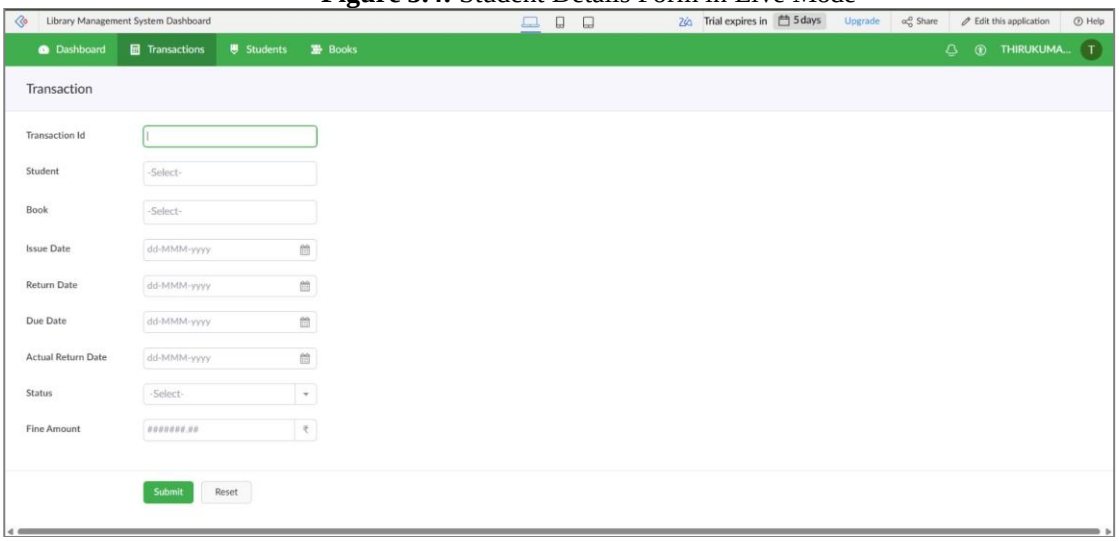



**Figure 5.5:** Book Issue Form in Live Mode

### 5.3 Reports

Five list reports present the data as simple tables. Each has a quick search box and column filters; the Book Issue report can also be filtered by Issue Status. The Overdue Books report uses the criteria *Issue_Status != Returned and Due_Date < today*, so only active past-due issues appear.

**Table 5.1:** Reports Implemented

| Report | Type | Source Form | Columns / Filter | Role |
|---|---|---|---|---|
| Book Details Report | List | Book_Details | Book ID, Name, Author, Category, Total and Available Quantity; search by name or category | Admin |
| Student Details Report | List | Student_Details | ID, Name, Department, Year, Email, Phone | Admin |
| Book Issue Report | List | Book_Issue | Issue ID, Student, Book, Issue Date, Due Date, Status; filter by status | Admin |
| Book Return Report | List | Book_Return | Return ID, Issue ID, Student, Book, Return Date, Fine, Status | Admin |
| Overdue Books Report | List | Book_Issue | Student, Book, dates, status; active issues past due date | Admin |

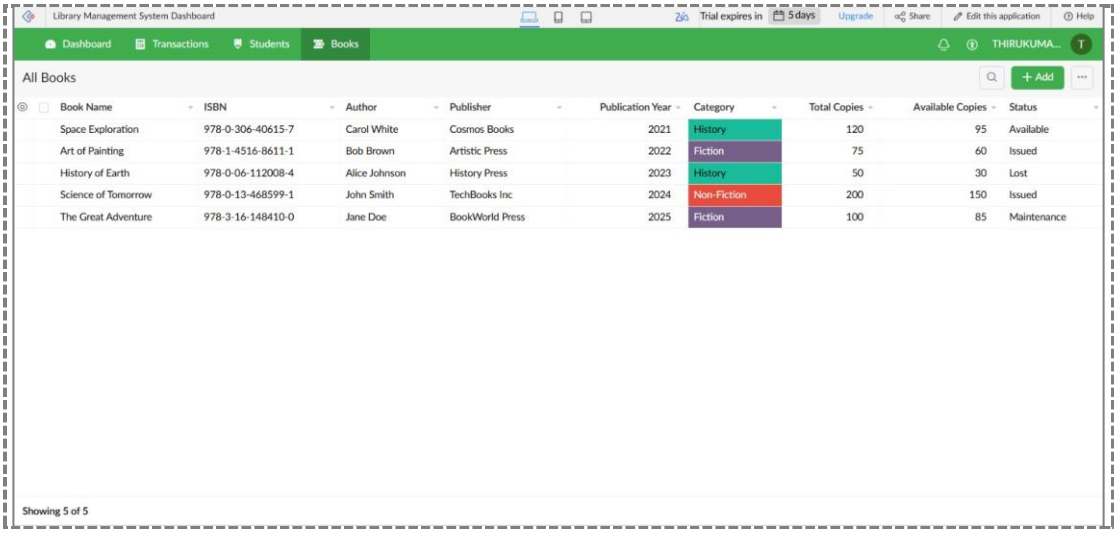



**Figure 5.7:** Book Details Report

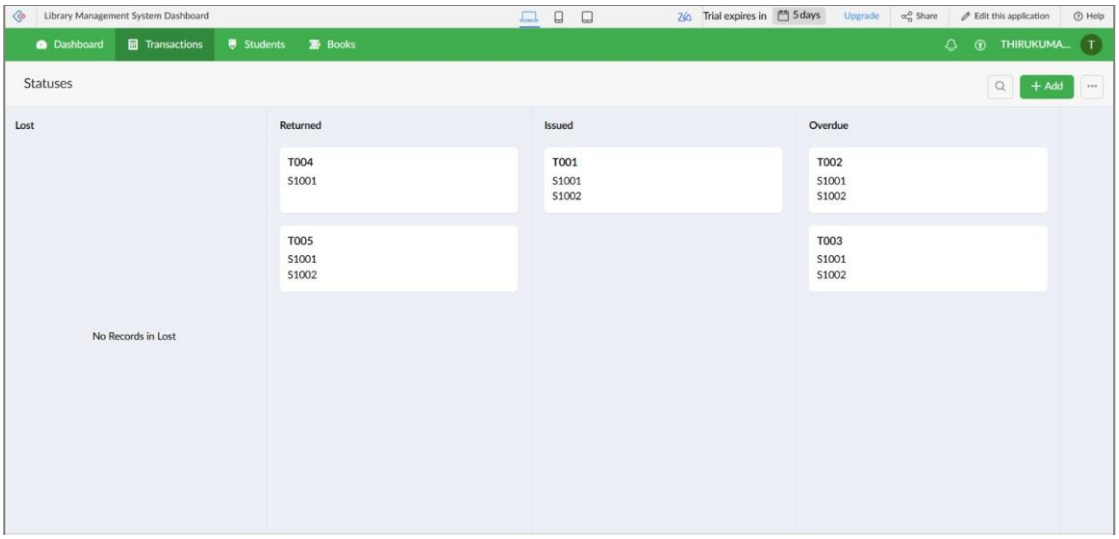



**Figure 5.8:** Overdue Books Report

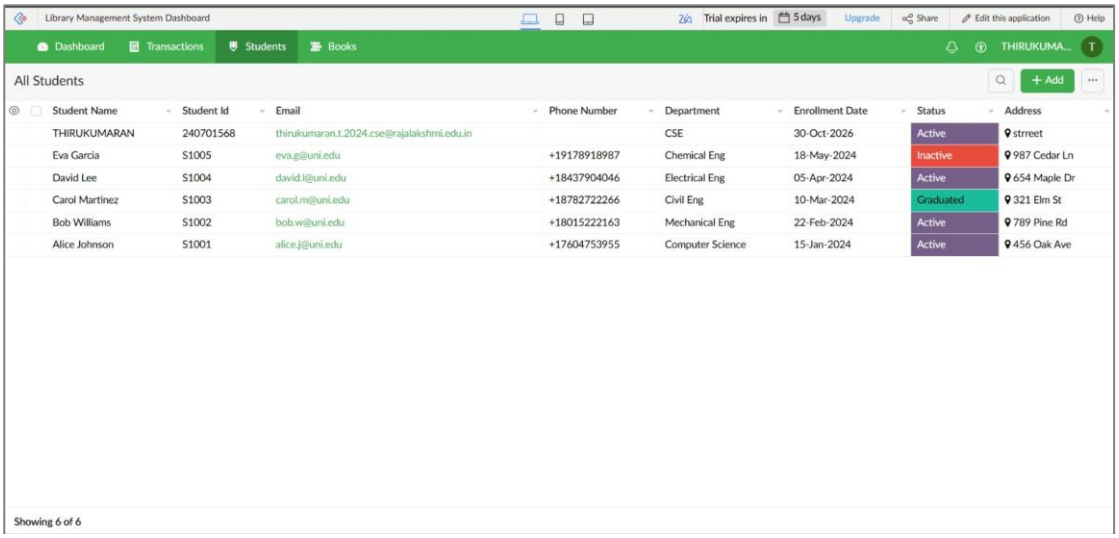



**Figure 5.8a:** Student Details Report

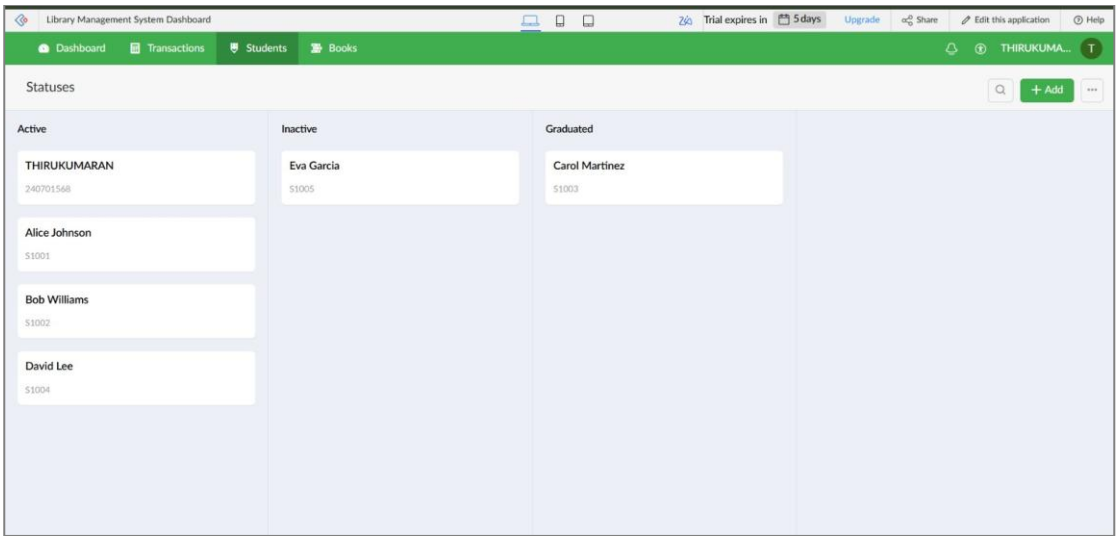



**Figure 5.8b:** Students by Status (Kanban View)

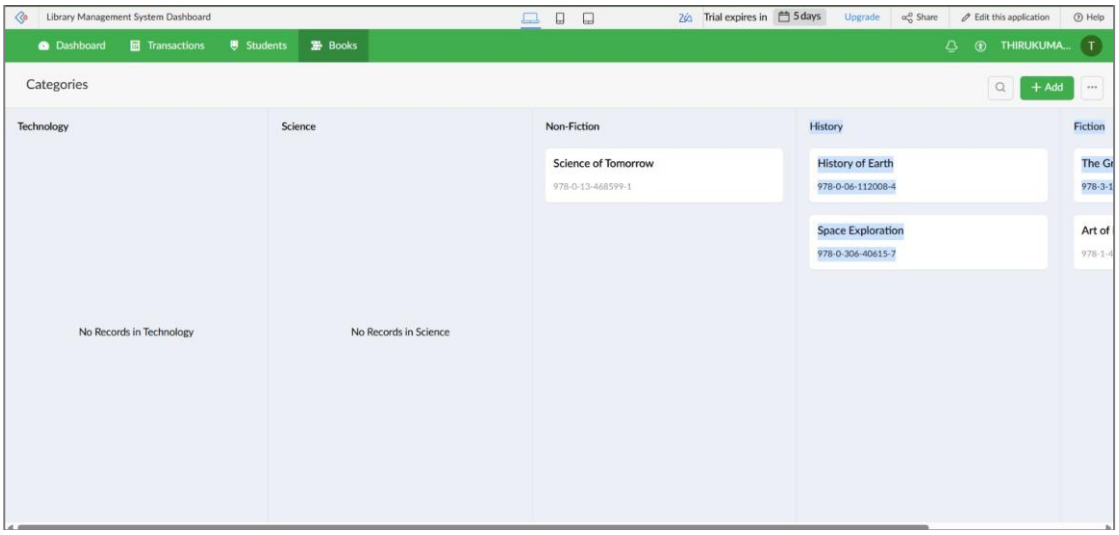



**Figure 5.8c:** Books by Category (Kanban View)

### 5.4 Pages and Dashboards

The Admin Dashboard page is intentionally plain. It shows four values read from the database — Total Books (count of Book Details), Total Students (count of Student Details), Books Issued (issue records with status Issued or Overdue) and Available Books (sum of Available Quantity) — and a Recent Transactions table with Student, Book, Issue Date, Return Date and Status. There are no charts, coloured cards, icons or quick-link tiles.

**Library Management System – Admin Dashboard**

Total Books: 4
Total Students: 4
Books Issued: 0
Available Books: 29

**Recent Transactions**

| Student | Book | Issue Date | Return Date | Status |
|---|---|---|---|---|
| | | | | |
| | | | | |

**Figure 5.9:** Admin Dashboard Layout with Sample Data (plain, no charts)

### 5.5 Deluge Scripting

Five Deluge scripts enforce the business rules. Validation scripts run *On Validate* and can stop the submission with *cancel submit*; stock scripts run *On Success*, after the record is saved. Each script fetches the related record by its ID, which keeps the lookup efficient. The complete listings are in Appendix C.

**Table 5.2:** Deluge Scripts Implemented

| ID | Location / Trigger | Purpose | Concepts Used |
|---|---|---|---|
| DS-01 | Book_Issue → On Validate | Block issue if no copies or Due Date < Issue Date | fetch, if, alert, cancel submit |
| DS-02 | Book_Issue → On Success | Reduce Available Quantity; set status Issued | record update, if-else |
| DS-03 | Book_Return → On Validate | Block repeat return of the same issue | fetch, if, alert, cancel submit |
| DS-04 | Book_Return → On Success | Restore stock, mark Returned, calculate fine | date functions, record update |
| DS-05 | Book_Details → On Validate | Keep quantities non-negative and consistent | if, alert, cancel submit |

```
// Script ID : DS-01
// Location  : Book_Issue -> On Validate
// Purpose   : Block the issue when no copy is available or dates are invalid
bk = Book_Details[ID == input.Book];
if(bk.Available_Quantity <= 0)
{
    alert "No copies available for " + bk.Book_Name;
    cancel submit;
}
if(input.Due_Date < input.Issue_Date)
{
    alert "Due Date cannot be earlier than Issue Date";
    cancel submit;
}
```

**Code 5.1: Issue validation (DS-01)**

DS-01 fetches the selected book and compares its Available Quantity with zero. If no copy is free, the alert is shown and the record is not saved. The second check rejects a Due Date earlier than the Issue Date.

```
// Script ID : DS-02
// Location  : Book_Issue -> On Success
// Purpose   : Reduce Available Quantity by one and set the status to Issued
bk = Book_Details[ID == input.Book];
bk.Available_Quantity = bk.Available_Quantity - 1;
if(bk.Available_Quantity == 0)
{
    bk.Book_Status = "Out of Stock";
}
else
{
    bk.Book_Status = "Available";
}
input.Issue_Status = "Issued";
```

**Code 5.2: Stock reduction on issue (DS-02)**

DS-02 runs after the issue is saved. It reduces the stock by one, sets Book Status to Out of Stock when the last copy is issued, and sets the issue status to Issued.

```
// Script ID : DS-03
// Location  : Book_Return -> On Validate
// Purpose   : Prevent the same issue record from being returned twice
iss = Book_Issue[ID == input.Issue_Record];
if(iss.Issue_Status == "Returned")
{
    alert "This issue record has already been returned";
    cancel submit;
}
if(input.Return_Date < iss.Issue_Date)
{
    alert "Return Date cannot be earlier than Issue Date";
    cancel submit;
}
```

**Code 5.3: Duplicate-return check (DS-03)**

DS-03 reads the status of the selected issue. A record already marked Returned cannot be returned again, and a Return Date before the Issue Date is rejected.

```
// Script ID : DS-04
// Location  : Book_Return -> On Success
// Purpose   : Restore stock, mark the issue Returned and calculate any fine
iss = Book_Issue[ID == input.Issue_Record];
bk  = Book_Details[ID == iss.Book];
bk.Available_Quantity = bk.Available_Quantity + 1;
bk.Book_Status = "Available";
iss.Issue_Status = "Returned";
fine_rate = 0.0;  // set to the administrator-approved per-day rate
if(fine_rate > 0 && input.Return_Date > iss.Due_Date)
{
    late_days = input.Return_Date.daysBetween(iss.Due_Date).abs();
    input.Fine_Amount = late_days * fine_rate;
}
else
{
    input.Fine_Amount = 0;
}
```

**Code 5.4: Return processing (DS-04)**

DS-04 increases the stock and marks the issue Returned. The fine is calculated only when a rate is configured and the return is late, because the per-day rate is an administrator decision that the requirements do not fix. The late days are taken as an absolute value so the order of the dates does not affect the result.

```
// Script ID : DS-05
// Location  : Book_Details -> On Validate
// Purpose   : Keep quantities consistent
if(input.Total_Quantity < 0 || input.Available_Quantity < 0)
{
    alert "Quantity must not be negative";
    cancel submit;
}
if(input.Available_Quantity > input.Total_Quantity)
{
    alert "Available Quantity cannot exceed Total Quantity";
    cancel submit;
}
```

**Code 5.5: Quantity validation (DS-05)**

DS-05 protects the book record: quantities cannot be negative and Available Quantity cannot exceed Total Quantity.

### 5.6 Workflows and Automation

All automation is implemented as form workflows triggered by form events. No schedule, approval or Blueprint workflow was needed for this basic scope.

**Table 5.3:** Workflows Implemented

| Workflow | Type | Trigger / Event | Condition | Action(s) | Script |
|---|---|---|---|---|---|
| Issue Validation | Form | On Validate | Available Quantity ≤ 0 or invalid dates | Block submission with message | DS-01 |
| Stock Update on Issue | Form | On Success | Issue saved | Reduce quantity; status Issued | DS-02 |
| Return Validation | Form | On Validate | Issue already Returned | Block duplicate return | DS-03 |
| Stock Update on Return | Form | On Success | Return saved | Restore quantity; fine; status Returned | DS-04 |
| Quantity Validation | Form | On Validate | Negative or excess quantity | Block submission | DS-05 |

### 5.7 Integrations

No integration is implemented because the application is a stand-alone library record system. Table 5.4 records this decision and the options kept open.

**Table 5.4:** Integrations Implemented

| Integration | Service / Endpoint | Method | Status / Purpose |
|---|---|---|---|
| Email notification | Zoho Mail / sendmail | Deluge sendmail task | Not implemented; planned for overdue reminders |
| External services | REST API / webhook | invokeurl | Not required in current scope |

### 5.8 User Management and Role-Based Access

A single Administrator profile has full access to the four forms, five reports and the dashboard, as in Table 4.8. Students are stored as records, not users, so no student can sign in. If a student portal is added later, a second profile limited to the student's own records can be created with a record-level criterion.

### 5.9 AI and Analytics Features

Zia generated the starting application from the prompt below, which asks for four forms, five reports, a minimal dashboard and the stock workflows. Because a prompt cannot guarantee every result, lookups, workflows and dashboard styling were verified manually, and any coloured widgets that appeared were removed. Analytics charts are deliberately not used, to keep the dashboard plain.

```
Create a basic Library Management System in Zoho Creator.
Forms: Book Details, Student Details, Book Issue, Book Return
(Auto Number IDs, Lookups between forms).
Reports: Book Details, Student Details, Book Issue, Book Return,
Overdue Books, with search and filtering.
Dashboard: Total Books, Total Students, Books Issued,
Available Books and a Recent Transactions table - plain text,
default colours, no charts or decorative widgets.
Workflows: issue checks and reduces quantity; return increases it
and cannot be repeated; validate quantities and dates; fine only
after a per-day rate is configured. Do not delete existing records.
```

**Code 5.6: Summary of the Zia prompt used**

### 5.10 Pro-Code Extension

No pro-code extension was used because form-event Deluge was sufficient. Zoho Catalyst, custom JavaScript widgets or the Creator REST API could serve a future student portal or barcode-scanning service.

### 5.11 Mobile and Portal Access

The application opens in the Zoho Creator mobile app without additional development, so the librarian can issue and return books from a phone. No customer portal is configured.

# CHAPTER 6
# TESTING AND DEBUGGING

## 6.1 Testing Strategy

Testing was carried out at four levels. **Unit testing** exercised each form with valid and invalid input. **Script testing** ran every Deluge script on its event with the sample books and students. **Integration testing** followed the issue–return workflow end to end and compared reports and dashboard totals with the stored records. **Acceptance testing** is carried out by the users in Section 6.5. Negative tests cover zero stock, duplicate returns, invalid dates and out-of-range quantities.

## 6.2 Test Cases

Table 6.1 covers every functional requirement. The first ten cases follow the testing checklist of the project guide; the rest add validation and navigation checks.

**Table 6.1:** Test Cases

| TC ID | FR Ref. | Test Description | Input / Steps | Expected Result | Status |
|---|---|---|---|---|---|
| TC-01 | FR-01 | Create a book | Add Java Programming, quantity 10 | Appears in Book Details report | Pending |
| TC-02 | FR-02 | Create a student | Add Arun Kumar, CSE, year 2 | Appears in Student Details report | Pending |
| TC-03 | FR-03 | Issue a book with stock | Issue Java Programming to Arun Kumar | Available Quantity falls by one | Pending |
| TC-04 | FR-03 | Issue a book with zero stock | Issue a title whose Available Quantity is 0 | Issue blocked with message | Pending |
| TC-05 | FR-04 | Due Date before Issue Date | Enter an earlier Due Date | Validation error shown | Pending |
| TC-06 | FR-05 | Return an issued book | Select the active issue; submit | Available Quantity rises by one | Pending |
| TC-07 | FR-05 | Return the same issue again | Submit a second return | Duplicate return blocked | Pending |
| TC-08 | FR-06 | Late return after fine rate set | Return after the Due Date | Fine = late days × rate | Pending |
| TC-09 | FR-07 | Open Overdue Books report | Open the report | Only active past-due issues listed | Pending |
| TC-10 | FR-08 | Compare dashboard with forms | Count records and sum quantities | Totals match; no charts or coloured cards | Pending |
| TC-11 | FR-04 | Negative quantity | Enter Total Quantity −1 | Validation error shown | Pending |
| TC-12 | FR-04 | Available above Total | Available 12, Total 10 | Validation error shown | Pending |
| TC-13 | FR-03 | Lookup selection | Open Student and Book lookups | Existing records listed; no free text | Pending |
| TC-14 | FR-07 | Search a report | Search Python in Book Details report | Matching record shown | Pending |
| TC-15 | FR-09 | Menu access | Open every menu item | All forms and reports open | Pending |

### 6.3 Debugging Deluge Scripts

Scripts were debugged with *info* statements to print fetched values, the execute option of the Deluge editor, and the error or alert messages shown when a form is submitted. For example, while testing TC-07 the value of *iss.Issue_Status* is printed to confirm that DS-04 has changed it to Returned, because DS-03 relies on that value to block a second return.

### 6.4 Defect Log

Table 6.2 lists the defects identified from the implementation notes of the project guide, which warns that generated output needs manual verification. Status is updated after verification.

**Table 6.2:** Defect Log

| Bug ID | Description | Root Cause | Fix Applied | Status |
|---|---|---|---|---|
| BUG-01 | Coloured widgets on dashboard | Zia default dashboard layout | Remove widgets manually; keep forms and data | Verify |
| BUG-02 | Fine not calculated | No per-day rate configured | Configure the approved rate before enabling | Verify |
| BUG-03 | Lookup not linked correctly | Generated output not exact | Reconfigure the lookup field | Verify |
| BUG-04 | Duplicate return accepted | Status not checked on submit | DS-03 added on Book_Return | Verify |

### 6.5 User Acceptance Testing

At least two users should operate the application in the administrator role and record feedback in Table 6.3.

**Table 6.3:** User Acceptance Feedback

| User | Role Tested | Feedback | Action Taken |
|---|---|---|---|
| [Name] | Administrator | [Feedback] | [Action] |
| [Name] | Administrator | [Feedback] | [Action] |

### 6.6 Test Data and Boundary Tests

The sample records of the project guide were used as test data (Tables 6.4 and 6.5). Table 6.6 lists the boundary and negative conditions that were chosen deliberately.

**Table 6.4:** Sample Book Records for Testing

| Book Name | Author | Category | Quantity |
|---|---|---|---|
| Java Programming | James Gosling | Programming | 10 |
| Python Basics | John Smith | Programming | 8 |
| Database Management | Abraham Silberschatz | Database | 5 |
| Computer Networks | Andrew Tanenbaum | Networking | 6 |

**Table 6.5:** Sample Student Records for Testing

| Student Name | Department | Year |
|---|---|---|
| Arun Kumar | CSE | 2 |
| Priya | IT | 3 |
| Karthik | CSE | 2 |
| Divya | ECE | 1 |

**Table 6.6:** Boundary and Negative Test Conditions

| Condition | Value Tested | Expected Behaviour |
|---|---|---|
| Last copy issued | Available Quantity 1 → 0 | Issue allowed; status Out of Stock |
| Issue with no copy | Available Quantity 0 | Blocked with message |
| Return on due date | Return Date = Due Date | No fine |
| Return one day late, no rate | Rate not configured | No fine; return accepted |
| Quantity below zero | Total Quantity −1 | Validation error |
| Available above Total | 12 against 10 | Validation error |

Sample records are replaced or removed before the application is published to Production.

# CHAPTER 7
# DEPLOYMENT, SECURITY AND GOVERNANCE

## 7.1 Environments and Deployment Strategy

The application is built and tested in the Development environment. After all test cases pass and the checklist in Table 7.1 is complete, it is published to Production, where the administrator uses it. Every release carries a short version note, and the previous published version can be restored if a change misbehaves. Sample records are removed or replaced before real data is entered.

**Table 7.1:** Deployment Checklist

| S.No. | Checklist Item | Done ( ) |
|---|---|---|
| 1 | All fifteen test cases executed in Development | [ ] |
| 2 | Sample/test data removed or isolated | [ ] |
| 3 | Administrator user and permissions configured in Production | [ ] |
| 4 | Per-day fine rate configured and approved by the project owner | [ ] |
| 5 | Workflows enabled | [ ] |
| 6 | Application published and URL verified | [ ] |
| 7 | Version / release notes recorded | [ ] |

## 7.2 Performance Optimisation

The application is small, so optimisation is mainly good scripting practice (Table 7.2).

**Table 7.2:** Performance Optimisation Techniques

| Technique Applied | Component | Benefit |
|---|---|---|
| Fetch by ID criteria rather than scanning all records | DS-01 to DS-04 | Single-record lookup, quick response |
| No fetch statement inside a loop | All scripts | Constant number of database calls |
| Show only required columns | Reports | Smaller, faster report pages |
| Default filter for active issues | Overdue Books report | Fewer records to load |

## 7.3 Security Features

**Table 7.3:** Security Controls Implemented

| Security Control | Where Applied | Threat Mitigated |
|---|---|---|
| Login-based access | Whole application | Unauthorised access to records |
| Mandatory fields and validations | All forms | Invalid or incomplete data |
| Lookups instead of typed names | Issue and Return forms | Wrong or mismatched records |
| Existing records preserved | Forms and workflows | Accidental deletion or overwriting |
| No hard-coded credentials | Deluge scripts | Credential leakage (no integrations used) |

### 7.4 Governance and Administration

Zoho Creator records who added or modified each record and when, giving an audit trail. Forms and fields follow a consistent *Form_Name* and *Field_Name* convention. Reports can be exported as a backup, and the administrator maintains the application, the user list and the fine rate.

# CHAPTER 8
# RESULTS AND DISCUSSION

## 8.1 Application Output

The scenario below follows the sample records through the application. Table 8.1 shows the stock and dashboard values after each step.

**Table 8.1:** Scenario with Sample Data

| Step | Action by Administrator | Result | Available Qty (Java) | Available Books |
|---|---|---|---|---|
| 1 | Open dashboard | Total Books 4, Students 4, Issued 0 | 10 | 29 |
| 2 | Issue Java Programming to Arun Kumar | Issue saved, status Issued | 9 | 28 |
| 3 | Return the book | Return saved, status Returned | 10 | 29 |
| 4 | Try to return it again | Blocked: already returned | 10 | 29 |
| 5 | Issue when Available Quantity is 0 | Blocked: no copies available | 0 | — |

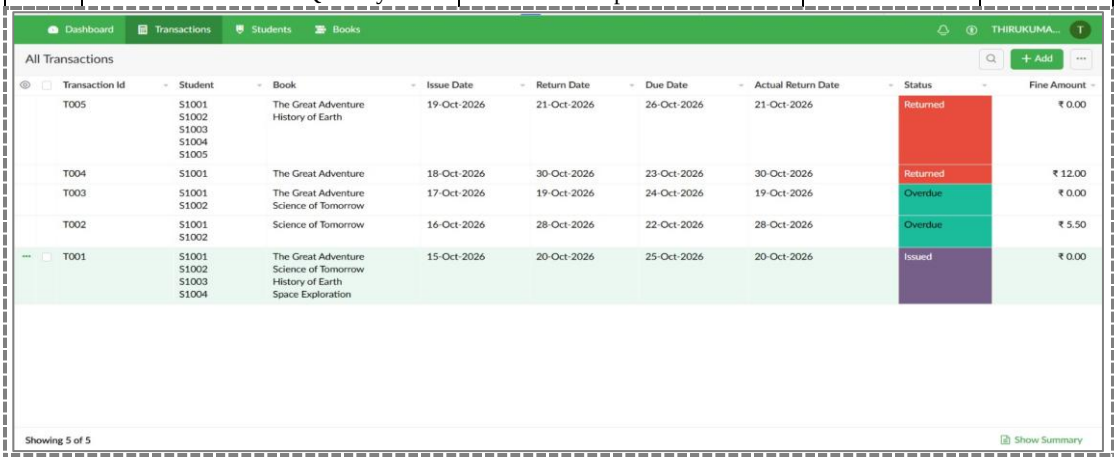



**Figure 8.1:** Scenario Step 1 – Issue a Book

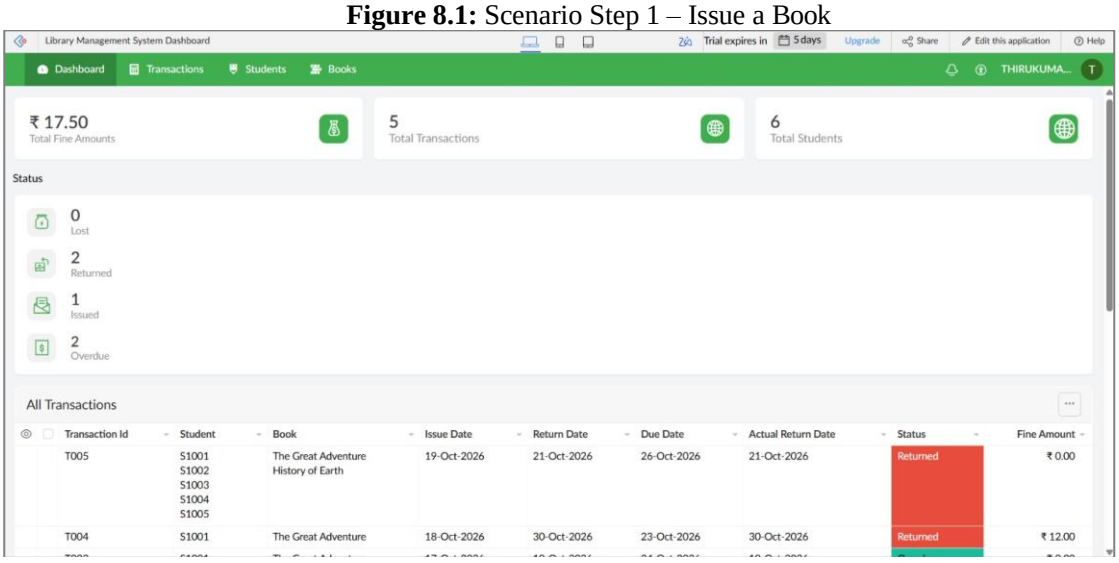



**Figure 8.2:** Scenario Step 2 – Return the Book

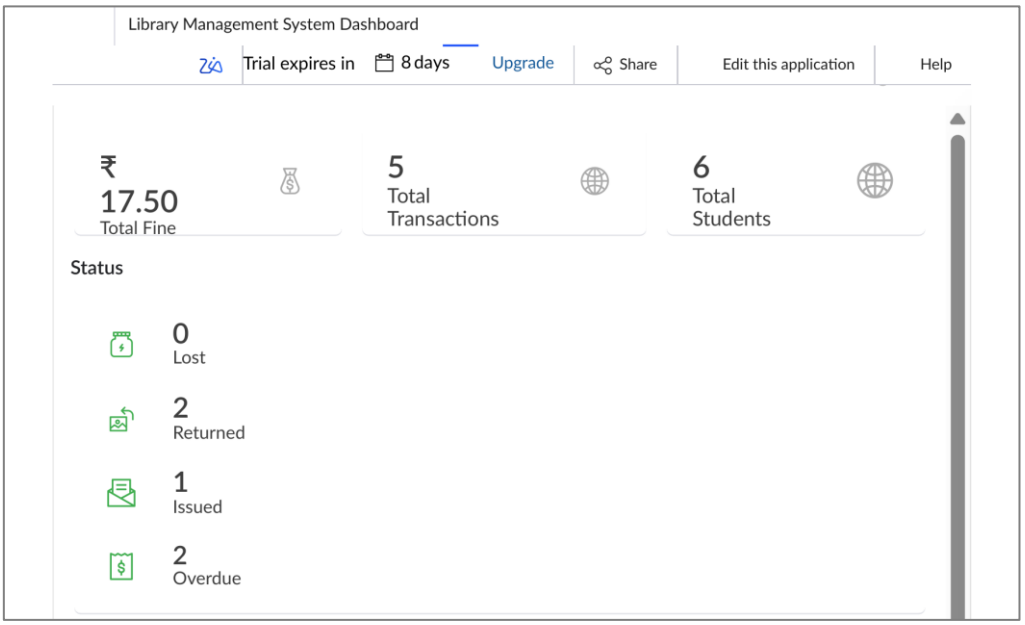



**Figure 8.3:** Scenario Step 3 – Dashboard Totals

## 8.2 Achievement of Objectives

**Table 8.2:** Mapping of Objectives to Outcomes

| Objective (Sec. 1.3) | Implemented Through | Outcome / Evidence |
|---|---|---|
| Forms and lookups | Sections 4.3, 5.2 | Four linked forms with Auto Number IDs |
| Automated stock, validation, fine | Sections 5.5, 5.6 | DS-01 to DS-05 enforce the rules |
| Reports and dashboard | Sections 5.3, 5.4 | Five reports and a plain dashboard |
| Zia AI app creation | Section 5.9 | Starting application generated and verified |
| Testing and deployment | Chapters 6, 7 | Fifteen test cases and a deployment checklist |

## 8.3 Comparison with the Existing System

**Table 8.3:** Existing System versus Proposed Application

| Parameter | Existing System | Proposed Application | Improvement |
|---|---|---|---|
| Processing an issue | Manual register entry and stock count | Two lookups and one save; stock updates itself | Fewer steps, no stock error |
| Overdue tracking | Manual date checking | Overdue Books report | Instant list |
| Report generation | Prepared by hand | Table reports with search | Always current |
| Data accessibility | One register or file | Any browser or mobile app | Anywhere access |

## 8.4 Limitations of the Low-Code Approach

Zia may not create every workflow, lookup or plain dashboard exactly as required, so manual verification and removal of unwanted widgets are needed. The fine rate is not fixed in the requirements and must be approved before automatic fines are enabled. Deeper customisation of the user interface is limited by the platform.

## 8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Forms, reports, the menu and the dashboard were **no-code** configuration. Stock, return and fine logic needed **low-code** Deluge. A student portal with barcode scanning or an external fine-payment service would need **pro-code** extension, and that is the point at which the project would move along the spectrum.

# CHAPTER 9
# CONCLUSION AND FUTURE ENHANCEMENTS

## 9.1 Conclusion

The Library Management System developed on Zoho Creator manages books, students, issues and returns through four linked forms. Deluge workflows check stock before every issue, reduce and restore Available Quantity, block duplicate returns, validate quantities and dates, and calculate overdue fines once a rate is configured.

Five table reports with search and filters and a plain Admin Dashboard give the librarian instant, accurate information drawn from the database. The project shows that a records-driven application can be built quickly with Zia and low-code tools, while remaining simple enough for a non-specialist to maintain.

## 9.2 Future Enhancements

- Scheduled workflow to mark overdue issues automatically and email reminders.
- Student login portal to view issued books and due dates.
- Barcode or ISBN scanning at the issue counter.
- Online payment of fines and a configurable fine-rate setting page.
- Analytics on popular categories and borrowing trends using Zoho Analytics.

## 9.3 Learning Outcomes and Career Relevance

The project built skills in data modelling with lookups, form validation, Deluge scripting, report design and AI-assisted app building. These skills relate directly to roles such as citizen developer, low-code developer, solutions consultant and automation engineer, and provide a base for moving later towards pro-code platforms such as Zoho Catalyst.

# REFERENCES


[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/

[3] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/

[4] R. Waszkowski, “Low-code platform for automating business processes in manufacturing,” *IFAC-PapersOnLine*, vol. 52, no. 10, pp. 376–381, 2019.

[5] A. Sahay, A. Indamutsa, D. Di Ruscio, and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in *Proc. 46th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA)*, 2020, pp. 171–178.

[6] Koha Community, “Koha Library Software.” [Online]. Available: https://koha-community.org/

# APPENDIX A

## APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Each feature is marked as implemented (Y) or not (N), with its place in the application and the report section that evidences it. Examiners can verify the table against the live application.

**Table A.1:** Feature Coverage Checklist

| Unit | Zoho Creator Feature | Done | Where in the Application | Section | Figure / Code |
|---|---|---|---|---|---|
| I | Application created; sections/menu organised | Y | Library Management System (Zia-built) | 5.1 | Fig. 5.1 |
| II | Forms with ≥ 8 distinct field types | Y | All four forms (9 field types) | 5.2 | Fig. 5.2–5.6 |
| II | Lookup field (relationship between forms) | Y | Book_Issue, Book_Return | 4.2, 5.2 | Fig. 4.2 |
| II | Subform (parent–child data) | N | Not required | — | — |
| II | Field validations / layout rules | Y | Quantities, dates, mandatory fields | 5.2, 5.5 | Code 5.1 |
| II | List reports with search and filters | Y | Five table reports | 5.3 | Table 5.1 |
| II | Kanban / Calendar / Pivot reports | N | Plain tables only, by design | 5.3 | — |
| II | Page / dashboard | Y | Admin Dashboard | 5.4 | Fig. 5.9 |
| II | Deluge scripting | Y | DS-01 to DS-05 | 5.5 | Code 5.1–5.5 |
| II | Form workflows (form events) | Y | Book_Details, Book_Issue, Book_Return | 5.6 | Table 5.3 |
| II | Schedule / Approval / Blueprint / Batch | N | Not in basic scope | 5.6 | — |
| III | Integration (CRM, REST, webhook) | N | Stand-alone application | 5.7 | Table 5.4 |
| III | Roles and permission profiles | Y | Administrator profile | 4.6, 5.8 | Table 4.8 |
| III | AI features (Zia app builder) | Y | Starting application generation | 5.9 | Code 5.6 |
| IV | Testing in Development and test cases | Y | Fifteen test cases | 6.2 | Table 6.1 |
| IV | Deployment checklist / publish | Y | Development → Production | 7.1 | Table 7.1 |
| IV | Security and governance controls | Y | Login, validations, audit trail | 7.3, 7.4 | Table 7.3 |
| V | Pro-code extension | N | Not used (justified) | 5.10 | — |
| V | Mobile access | Y | Zoho Creator mobile app (default) | 5.11 | Fig. 5.14 |

# APPENDIX B

## APPENDIX B – COURSE OUTCOME MAPPING

The table shows where each Course Outcome of CB23F35 is evidenced in this report. The self-assessed level is 1 (Low), 2 (Medium) or 3 (High); the examiner may revise it.

**Table B.1:** Mapping of Course Outcomes to Report Chapters

| CO | Course Outcome Statement | Evidenced In | Self-Assessed Level | Examiner Level |
|---|---|---|---|---|
| CO1 | Explain the principles, architecture, and business value of low-code application development using Zoho Creator. | Ch. 1, 2 | [ ] | |
| CO2 | Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting. | Ch. 4, 5 (5.1–5.6), 6 | [ ] | |
| CO3 | Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security. | Ch. 4.6, 5.7–5.9 | [ ] | |
| CO4 | Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices. | Ch. 7 | [ ] | |
| CO5 | Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer. | Ch. 5.9, 5.10, 8.4, 8.5, 9.3 | [ ] | |

# APPENDIX C

## APPENDIX C – COMPLETE DELUGE SOURCE CODE

Every Deluge script of the application is listed in full, grouped by form, with its Script ID, trigger and purpose.

### C.1 Book_Details Form

```
// Script ID : DS-05
// Location  : Book_Details -> On Validate
// Purpose   : Keep quantities consistent
if(input.Total_Quantity < 0 || input.Available_Quantity < 0)
{
    alert "Quantity must not be negative";
    cancel submit;
}
if(input.Available_Quantity > input.Total_Quantity)
{
    alert "Available Quantity cannot exceed Total Quantity";
    cancel submit;
}
```

**DS-05**

### C.2 Book_Issue Form

```
// Script ID : DS-01
// Location  : Book_Issue -> On Validate
// Purpose   : Block the issue when no copy is available or dates are invalid
bk = Book_Details[ID == input.Book];
if(bk.Available_Quantity <= 0)
{
    alert "No copies available for " + bk.Book_Name;
    cancel submit;
}
if(input.Due_Date < input.Issue_Date)
{
    alert "Due Date cannot be earlier than Issue Date";
    cancel submit;
}
```

**DS-01**

```
// Script ID : DS-02
// Location  : Book_Issue -> On Success
// Purpose   : Reduce Available Quantity by one and set the status to Issued
bk = Book_Details[ID == input.Book];
bk.Available_Quantity = bk.Available_Quantity - 1;
if(bk.Available_Quantity == 0)
{
    bk.Book_Status = "Out of Stock";
}
else
{
    bk.Book_Status = "Available";
}
input.Issue_Status = "Issued";
```

**DS-02**

## C.3 Book_Return Form

```
// Script ID : DS-03
// Location  : Book_Return -> On Validate
// Purpose   : Prevent the same issue record from being returned twice
iss = Book_Issue[ID == input.Issue_Record];
if(iss.Issue_Status == "Returned")
{
    alert "This issue record has already been returned";
    cancel submit;
}
if(input.Return_Date < iss.Issue_Date)
{
    alert "Return Date cannot be earlier than Issue Date";
    cancel submit;
}
```

**DS-03**

```
// Script ID : DS-04
// Location  : Book_Return -> On Success
// Purpose   : Restore stock, mark the issue Returned and calculate any fine
iss = Book_Issue[ID == input.Issue_Record];
bk  = Book_Details[ID == iss.Book];
bk.Available_Quantity = bk.Available_Quantity + 1;
bk.Book_Status = "Available";
iss.Issue_Status = "Returned";
fine_rate = 0.0;  // set to the administrator-approved per-day rate
if(fine_rate > 0 && input.Return_Date > iss.Due_Date)
{
    late_days = input.Return_Date.daysBetween(iss.Due_Date).abs();
    input.Fine_Amount = late_days * fine_rate;
}
else
{
    input.Fine_Amount = 0;
}
```

**DS-04**

# APPENDIX D

## APPENDIX D – USER MANUAL

### D.1 Accessing the Application

Open the application link in a browser, or open the Zoho Creator mobile app, and sign in with the administrator's Zoho account. The Library Management System appears with the menu on the left: Dashboard, Book Details, Student Details, Book Issue, Book Return and Reports.

### D.2 Instructions for the Administrator

**Add a book.**

- Open *Book Details* and click *Add.*
- Enter Book Name, Author, Category, Publisher and ISBN.
- Enter Total Quantity and Available Quantity (equal for a new book), then click *Submit.*

**Add a student.**

- Open *Student Details* and click *Add*.
- Enter name, department, year, email and phone, then *Submit.*

**Issue a book.**

- Open *Book Issue* and click *Add.*
- Choose the Student and the Book from the lookups and set the Due Date.
- Click *Submit*. If no copy is available the form shows a message and does not save.

**Return a book.**

- Open *Book Return* and click *Add.*
- Select the active Issue Record and enter the Return Date, then *Submit.*
- The stock increases by one; a fine appears only if the return is late and a rate is set.

**View reports and the dashboard.**

- Open *Reports* to see the five table reports; use the search box or filters.
- Open *Overdue Books* for active issues past their due date.
- Open *Dashboard* for totals and recent transactions.

### D.3 Messages and What to Do

**Table D.1:** Messages Shown by the Application

| Message | Meaning | Action |
|---|---|---|
| No copies available for <book> | Available Quantity is zero | Wait for a return or choose another book |
| Due Date cannot be earlier than Issue Date | Dates entered in the wrong order | Correct the Due Date |
| This issue record has already been returned | The return was entered before | Choose a different active issue |
| Quantity must not be negative | Quantity below zero | Enter zero or a positive number |
| Available Quantity cannot exceed Total Quantity | Available is larger than Total | Correct either quantity |

## D.4 Good Practices

- Always choose students and books from the lookups; never type names.
- Enter a return as soon as the book is received so that stock stays correct.
- Do not delete issue records; mark them Returned so history is kept.
- Ask the project owner to approve the per-day fine rate before late returns are processed.
- Check the dashboard totals against the reports at the end of each day.

# APPENDIX E

## APPENDIX E – EVALUATION RUBRIC

*(For Examiner Use Only)*

**Table E.1:** Evaluation Rubric for the Individual Project

| S.No. | Criterion | Indicators | Max. Marks | Marks Awarded |
|---|---|---|---|---|
| 1 | Problem Definition and Requirement Analysis | Clarity of problem, measurable objectives, complete FR/NFR, use cases | 10 | |
| 2 | System Design | Architecture, ER/data model, form specifications, workflow design, permission matrix | 10 | |
| 3 | Implementation | Forms, reports, pages, Deluge scripts, workflows, roles | 30 | |
| 4 | Testing and Debugging | Test cases, defect log, debugging evidence | 10 | |
| 5 | Deployment, Security and Governance | Environments, security controls, governance | 10 | |
| 6 | Report and Presentation | Format, clarity, figures, references | 10 | |
| 7 | Viva-Voce | Understanding and demonstration of the live application | 20 | |
| | Total | | 100 | |

Student: **Thirukumaran T** Register No.: **240701568** Project: **Library Management System**

# APPENDIX F

## APPENDIX F – VIVA-VOCE RECORD SHEET

**Table F.1:** Viva-Voce Questions and Responses

| S.No. | Question Asked | Response Summary | Marks |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |
| 4 | | | |
| 5 | | | |

**Table F.2:** Marks Summary

| Component | Maximum | Marks Awarded |
|---|---|---|
| Report evaluation | 60 | |
| Live demonstration and viva-voce | 40 | |
| Total | 100 | |

**INTERNAL EXAMINER** **EXTERNAL EXAMINER**